

Fast Squaring Algorithm Design and Complexity Numerical Analyses for Public-Key Cryptosystems

Chia-Long Wu*, Der-Chyuan Lou, and Te-Jen Chang

**Department of Aviation and Communication Electronics
Chinese Air Force Institute of Technology
chialongwu@seed.net.tw*

*Department of Electrical Engineering
Chung Cheng Institute of Technology, National Defense University
dclouprof@gmail.com*

Abstract

Modular exponentiation is an important operation in several public-key cryptography. The standard squaring algorithm is well-known and the Guajardo-Paar algorithm fixes the improper carry handling bug produced by the standard squaring algorithm, but produces error-indexing bug. In this paper, one squaring algorithm is proposed, which stores base products into the Look-Up Table before the squaring computation and base size comparison method. The proposed algorithm can not only evaluate squaring efficiently but also avoid bugs produced in other proposed algorithms (the Guajardo-Paar algorithm and the Yang-Heih-Laih algorithm). The performance of the proposed algorithm is 1.615 times faster than the standard squaring algorithm and much faster than others' algorithms.

Keywords: *Number theory, complexity analysis, algorithm design, modular multiplication, public-key cryptosystem.*

1. Introduction

Public-key cryptographic systems, such as the RSA [1-2] often involve raising large elements of some group fields to a large power. Some methods of fast modular exponentiation have been proposed and applied to the above mentioned cryptosystems in past years.

Efficient squarings and multiplications algorithms play important roles in very large integer arithmetic. Some squaring methods are strictly faster than many multiplication methods [3-7]. The standard squaring algorithm [3] is quite well-known, but there is an improper carry handling bug in this algorithm. The Guajardo-Paar algorithm [6] fixes the improper carry handling bug, but produces error-indexing bug. In this paper, one squaring algorithm is proposed which can avoid both improper carry handling bug produced by the standard squaring algorithm [3] and the error-indexing bug produced by the Guajardo-Paar algorithm [5]. For base b , the products of $(x_i * x_j)_b$ can be pre-computed,

where $1 \leq x_i, x_j \leq (b-1)$, that is, $1*1, 1*2, 1*3, \dots$, and $(b-1)(b-1)$ are pre-computed respectively. These products will be determined in advance and stored into the Look-Up Table, and then we use base size comparison method to solve the bug problems produced by the standard squaring algorithm and the Guajardo Paar algorithm. The proposed algorithm is about 1.77 times and 1.84 times faster than the Yang-Hseih-Laih algorithm and the Guajardo-Parr algorithm respectively. Moreover, the proposed algorithm not only avoids the bugs in the standard squaring algorithm and the Guajardo-Paar algorithm but also improves the performance of the Yang-Hseih-Laih algorithm.

The rest of this paper is organized as follows. In Section 2, we review three squaring algorithms including the standard squaring algorithm, the Guajardo-Paar algorithm, and the Yang-Heih-Laih algorithm. In Section 3, we present one squaring algorithm which can avoid the bugs mentioned in the standard squaring algorithm and the Guajardo-Paar algorithm [3, 6] and have a better performance than Yang-Heih-Laih algorithm [7]. The complexity analyses are described in Section 4. Finally, we give the conclusions and future works in Section 5.

2. Review of Previous Works

The standard squaring algorithm, the Guajardo-Paar algorithm, and the Yang-Hseih-Laih algorithm are three famous methods for speeding up the computation of squaring. In the following subsections, we will introduce these squaring algorithms.

The Standard Squaring Algorithm

Squaring is used to substitute for multiplication because squaring is much more efficient than multiplication [7-11]. This is due to the fact that $s_{ij} = x_i * x_j = s_{ji}$. The standard squaring procedures are described in Figure 1 and in Algorithm 1 [3]. Thus, we can modify the standard multiplication

procedures to take advantage of this property of the squaring operation.

Algorithm 1 (The standard squaring algorithm)

Input: Integer $X = (x_n x_{n-1} \dots x_2 x_1)_b$

Output: Integer $S = X^2 = (s_{2n} s_{2n-1} \dots s_2 s_1)_b$

begin

$s_i = 0$ for $i = 1$ to $2n$

for $i = 1$ to n

$(u, v)_b = s_{i+i-1} + x_i * x_i$

$s_{i+i-1} = v$

for $j = i+1$ to n

$(u, v)_b = s_{i+j-1} + 2 * x_j * x_i + u$

$s_{i+j-1} = v$

endfor

$s_{i+n} = u$

endfor

Return $S = (s_{2n} s_{2n-1} \dots s_1)_b$

end.

Example 1:

Let $X = 432$, i.e., $n = 3$ and $b = 10$. The squaring procedures of X using Algorithm 1 are shown in Table 1.

However, we find that the carry-sum pair produced by operation:

$$(u, v)_b = s_{i+j} + 2 * x_j * x_i + u$$

in Algorithm 1 may be 1 bit longer than a single-precision number which requires w bits. Since

$$(2^w - 1) + 2 * (2^w - 1) * (2^w - 1) + (2^w - 1) = 2^{2w+1} - 2^{w+1},$$

and

$$2^{2w} - 1 < 2^{2w+1} - 2^{w+1} < 2^{2w+1} - 1,$$

the carry-sum pair requires $2w+1$ bits instead of $2w$ bits for its representation. Thus, we need to accommodate this 'extra' bit during the execution of the operation in Algorithm 1. The resolution of this carry may depend on the carry bits handled by the particular processor's architecture.

The Guajardo-Paar Algorithm

This algorithm fixes the improper carry handling bug produced in the standard squaring algorithm [6, 12-16]. This algorithm is demonstrated in Algorithm 2. It's mainly used to prevent the improper carry handling bug. This algorithm does maintain accuracy in carry handling.

Algorithm 2 (The Guajardo-Parr algorithm)

Input: Integer $X = (x_n x_{n-1} \dots x_2 x_1)_b$

Output: Integer $S = X^2 = (s_{2n} s_{2n-1} \dots s_2 s_1)_b$

begin

$s_i = 0$ for $i = 1$ to $2n$

for $i = 1$ to n

$(u, v)_b = s_{2i-1} + x_i^2$

$s_{2i-1} = v, d = u, e = 0$

for $j = i+1$ to n

$(p, q) = x_i * x_j$

$(u, v)_b = s_{i+j-1} + (p, q) + d, s_{i+j-1} = v,$

$d = u$

$(u, v)_b = s_{i+j-1} + (p, q) + e, s_{i+j-1} = v,$

$e = u$

endfor

$(u, v)_b = d + e, d = v, e = u$

$(u, v)_b = s_{i+n} + d, s_{i+n} = v$

$s_{i+n+1} = e + u$

endfor

Return $S = (s_{2n} s_{2n-1} \dots s_2 s_1)_b$

end.

Let $X = (x_n x_{n-1} \dots x_2 x_1)_b$ be a multiprecision integer in base- b representation. Then $S = X^2 = (s_{2n} s_{2n-1} \dots s_2 s_1)_b$ can be computed using the following algorithm [6].

Example 2:

Let $X = 876$, i.e., $n = 3$ and $b = 10$. The squaring procedures of X using Algorithm 2 are shown in Table 2.

The Yang-Hseih-Laih Algorithm

The Yang-Hseih-Laih algorithm [16] avoids both the improper carry handling bug [3] produced by Algorithm 1 and the error-indexing bug [6] produced by Algorithm 2. The Yang-Hseih-Laih algorithm is described as Algorithm 3.

In the Yang-Hseih-Laih algorithm, the SPMs (Single Precision Multiplications) of Algorithm 1 are divided into two parts: $x_i * x_j$ and $x_j * x_i$. Compute these two parts separately and then combine these two results. Because the second part should be doubled, if it is firstly computed, and then double together, the improper carry bug is disappeared. Therefore Yang-Hseih-Laih algorithm firstly compute the multiplication $x_j * x_i$ ($i \neq j$), and then double this part by shifting one bit in Algorithm 3.

Algorithm 3 (The Yang-Hseih-Laih algorithm)

Input: Integer $X = (x_n x_{n-1} \dots x_2 x_1)_b$

Output: Integer $S = X^2 = (s_{2n} s_{2n-1} \dots s_2 s_1)_b$

begin

$s_i = 0$ for $i = 1$ to $2n$

for $i = 1$ to n

$u = 0$

for $j = i+1$ to n

$(u, v)_b = s_{i+j} + x_j * x_i + u$

```

     $s_{i+j-1} = v$ 
endfor
     $s_{i+n} = u$ 
     $s = 2s$  (Shift  $s$  left 1 bit)
     $u = 0$ 
endfor
for  $i = 1$  to  $n$ 
     $(u, v)_b = s_{2i} + x_i * x_i + u, s_{2i} = v$ 
     $(u, v)_b = s_{2i+1} + u, s_{2i+1} = v$ 
endfor
Return  $S = (s_{2n} s_{2n-1} \dots s_2 s_1)_b$ 
end.

```

All the SPSs (Single Precision Shifts) of Algorithm 1 are handled in Algorithm 3. Therefore, there is no carry propagation in Algorithm 3. This strategy eliminates all the extra carry propagation of $2 * x_j * x_i$ without any extra register or any operation. Yang, Hseih, and Laih propose this algorithm to fix these bugs as shown in Algorithm 3. It needs $(n^2 + n) / 2$ SPMs, $2n^2 + 2n$ SPAs (Single Precision Additions), and $2n^2 + 3n + 4$ assignments in Algorithm 3.

Example 3:

Let $X = 987$, i.e., $n = 3$ and $b = 10$. The squaring procedures of X using Algorithm 3 are shown in Table 3.

3. The Proposed Algorithm

In this section, one algorithm is proposed that not only to avoid both the improper carry handling bug produced by Algorithm 1 [3] and the error-indexing bug produced by Algorithm 2 [6], but also to improve the performance of the three squaring algorithms depicted in Section 2. Firstly, we discuss the improper carry handling bug and error-indexing bug respectively. Then, the proposed method is detailed described and analyzed in Algorithm 4.

➤ **Improper Carry Handling Bug**

We assume $(u, v) = s_{i+j} + 2 * x_j * x_i + u$, in Algorithm 1 the probabilistic distribution of the four parameters s_{i+j}, x_i, x_j, u , are all discrete uniform distribution. The calculation result of the middle term $2 * x_j * x_i$ dominates the accuracy of the whole calculation. Therefore, considering $2 * x_j * x_i$ being 65 bits, the products $x_j * x_i$ only needs to be 64 bits, and then the overflow will occur. So, the resolution of this carry may depend on the carry bits handled by the particular processor's architecture in Algorithm 1.

➤ **Error Indexing Bug**

In Algorithm 2, we assume i as the upper bound of

the n of the outer loop. We will find the index of s in " $s_{i+n+1} = e + u$ " is $2n+1$. Note that the definition scope " $S = (s_{2n} s_{2n-1} \dots s_2 s_1)_b$ " of the index is from 1 to $2n$. Therefore, the error indexing bug will be produced in Algorithm 2. The error indexing bug can be resolved by Algorithm 3.

Example 4:

Let $X=789$, i.e., $n=3$ and $b=10$. The squaring procedures of X using Algorithm 4 are shown in Table 4.

In order to fix the improper carry handling bug produced by Algorithm 1 and the error-indexing bug produced by Algorithm 2, the proposed method permits realization on a microcontroller without a subtract-with-borrow instruction and uses base size comparison method in Algorithm 4 to eliminate the bugs produced by Algorithm 1 and Algorithm 2.

At first, we compute the products of $1*1, 1*2, \dots, 1*(b-1), 2*1, 2*2, \dots$, and $(b-1)(b-1)$ in advance and store the products $1, 2, \dots, (b-1), 2, 4, \dots$, and $(b-1)^2$ in the LUT. The numbers of the products are at most $(b-1)^2$. If b is far smaller than n , where b is the base of the integer and n is the digit-length of the integer, the storage space used is very small and the computational time for the products can be omitted. So, there are only $2n^2$ SPAs and $3n$ assignments in Algorithm 4. The other analyses in further detail are discussed in Section 4 and shown in Table 5.

Algorithm 4 (Proposed algorithm)

Input: Integer $X = (x_n x_{n-1} x_{n-2} \dots x_2 x_1)_b$

Output: Integer $S = X^2 = (s_{2n} s_{2n-1} s_{2n-2} \dots s_2 s_1)_b$

begin

Compute the products of $1*1, 1*2, \dots, 1*(b-1), 2*1, 2*2, \dots$, and $(b-1)(b-1)$ respectively and store these results along with the corresponding products $1, 2, \dots, (b-1), 2, 4, \dots$, and $(b-1)^2$ in the LUT (Look-Up Table).

$s_i = 0$ for $i=1$ to $2n$

for $i=1$ to n

for $j=1$ to n

$(u, v)_b = x_i * x_j$ (Get the value $x_i * x_j$ from the LUT)

$s_{i+j} = s_{i+j} + u$

$s_{i+j-1} = s_{i+j-1} + v$

if $s_{i+j-1} \geq b$

then $s_{i+j} = s_{i+j} + 1$

$s_{i+j-1} = s_{i+j-1} - b$

else $s_{i+j-1} = s_{i+j-1}$

if $s_{i+j} \geq b$

```

    then  $s_{i+j+1} = s_{i+j+1} + 1$ 
         $s_{i+j} = s_{i+j} - b$ 
    else  $s_{i+j} = s_{i+j}$ 
endfor
endfor
Return  $S = (s_{2n}s_{2n-1}s_{2n-2}\dots s_2s_1)_b$ 
end.

```

4. Complexity Analyses

The numbers of single precision operations of Algorithm 1, 2, 3, and 4 are shown in Table 3. Because the SPM is the most time-consuming operation in Algorithm 1, 2, and 3, we can store the results of the SPMs into LUT (Look-Up Table) in advance to reduce the time complexity, which is like discussed in Section 3. Because we compute the products of $1*1$, $1*2$, ..., $1*(b-1)$, $2*1$, $2*2$, ..., and $(b-1)(b-1)$ in advance and store the products $1, 2, \dots, (b-1)$, $2, 4, \dots$, and $(b-1)^2$ in the LUT, the space for the products is $(b^4 + 2b^3 + b^2)/4$. If b is far smaller than n , where b is the base of the integer and n is the digit-length of the integer, the storage space can be assumed very small and the computational time for the products can be omitted. The longer the length of the integer n in comparison with the length of the base b , the less storage space the numbers of the products. So we only consider the effects of SPA, SPS, and assignments for performance estimation in Algorithm 4.

Suppose that the execution-time weight of an assignment is 1 and the execution-time weight can be estimated as 2, 3, and 11 for SPA, SPS, and SPM respectively [16]. Here the execution-time weight of SPM is used only for Algorithm 1, 2, and 3. According to these estimations, we can analyze Algorithm 1, 2, 3, and 4 for specific execution time weight in Table 5. If we assume the speedup ratio of Algorithm 1 is 1.00, the speedup ratios for Algorithm 1, 2, 3, and 4 are shown in Table 6. The numbers of different digits for Algorithm 1, 2, 3, and 4 in Table 5 are executed as shown in Table 7.

The analyses of squaring algorithms for specific execution time weight are shown in Table 5. Most importantly, we should notice that the execution time of the single precision operations depends on the operands. For example, the clocks of SPA are 1, 2, or 3 clocks for register-to-register, memory-to-register, and register-to-memory respectively [16-18]. That mainly causes the different performance of the analyses among Algorithm 1, 2, 3, and 4.

5. Conclusions

This paper introduces one squaring algorithm, which is accurate and efficient in comparison with the standard squaring algorithm, the Guajardo-Paar algorithm, and

Yang-Hsieh-Laih algorithm. The performance of the proposed algorithm is 1.615, 1.77, and 1.84 times faster than squaring computation by using the standard squaring algorithm, the Yang-Hsieh-Laih algorithm, and the Guajardo-Paar algorithm. Moreover, this algorithm can efficiently apply to fast modular arithmetic in cryptographic applications.

The proposed algorithm avoids the bugs among the standard squaring algorithm, the Guajardo-Paar algorithm, and Yang-Hsieh-Laih algorithm and improves the performance in squaring computation. So the proposed method is superior to Algorithm 1, 2, and 3. In the future, we intend to investigate the impact of the operand size and partition size on the area and time requirements [19-24].

References

- [1] D. E. Knuth, *The Art of Computer Programming*, vol. 2, *Seminumerical Algorithms*, Addison-Wesley, 3rd Edition, 1998.
- [2] A. J. Menezes, P. C. Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1997.
- [3] C. K. Koc, Tech. Notes, "High-Speed RSA Implementation," RSA Labs. Tech. Note TR 201, <http://www.rsasecurity.com/rsalabs/tech-notes>, August 14, 2007.
- [4] D. Zuras, "On squaring and multiplying large integers," *Proceedings of 11th Symposium on Computer Arithmetic*, vol. 29, pp. 260-271, June 1993.
- [5] D. Zuras, "More on squaring and multiplying large integers," *IEEE Transactions on Computers*, vol. 43, no. 8, pp. 899-908, 1994.
- [6] J. Guajardo and C. Paar, "Modified squaring algorithm," <http://citeseer.ist.psu.edu/672729.html>, 1999.
- [7] P.-Y. Hsieh and C.-S. Lai, "An exception handling model and its application to the multiple-precision integer library," *Thesis for Master of Science*, June 2003.
- [8] A. Karatsuba and Y. Ofman, "Multiplication of multidigit numbers on automata," *Soviet Physics-Doklady*, vol. 7, pp. 595-596, 1963.
- [9] A. L. Toom, "The complexity of a scheme of functional elements realizing the multiplication of integers," *Soviet Mathematics*, vol. 3, pp. 714-716, 1963.
- [10] J. Bos and M. J. Coster, "Addition chain heuristics," *Proceedings of Crypto '89, Jan. 1989*, pp. 400-407.
- [11] G. W. Reitwiesner, *Binary arithmetic*, *Advances in Computers*, Academic Press, vol. 1, Jan. 1960, pp. 231-308.
- [12] J. Jedwab and C. J. Mitchell, "Minimum weight

- modified signed-digit representations and fast exponentiation,” *IEE Electronic Letters*, vol. 25, no. 17, pp. 1171-1172, 1989.
- [13] S. Arno and F. S. Wheeler, “Signed digit representations of minimal hamming weight,” *IEEE Transactions on Computers*, vol. 42, no. 8, pp. 1007-1010, 1993.
- [14] D. M. Gordon, “A survey of fast exponentiation methods,” *Journal of Algorithms*, vol. 27, no. 1, pp. 129-146, 1998.
- [15] M. Joye and S.-M. Yen, “Optimal left-to-right binary signed-digit recoding,” *IEEE Transactions on Computers*, vol. 49, no. 7, pp. 740-748, 2000.
- [16] W.-C. Yang, P.-Y. Hesih, and C.-S. Lai, “Efficient squaring of large integers,” *IEICE Transactions on Fundamentals*, vol. E87-A, no. 5, pp. 1189-1192, 2004.
- [17] N. Rampersad, J. Shallit, and M.-W. Wang, “Avoiding large squares in infinite binary words,” *Theoretical Computer Science*, vol. 339, no. 1, pp. 19-34, June 2005.
- [18] A. Y. Ãzban, “New methods for approximating square roots,” *Applied Mathematics and Computation*, vol. 175, no. 1, pp. 532-540, 2006.
- [19] B. H. Gross, “An elliptic curve test for Mersenne primes,” *Journal of Number Theory*, vol. 110, no. 1, pp. 114-119, 2005.
- [20] D. Bakalis, K. D. Adaos, D. Lymperopoulos, M. Bellos, H. T. Vergos, G. P. Alexiou, and D. Nikolos, “A core generator for arithmetic cores and testing structures with a network interface,” *Journal of Systems Architecture*, vol. 52, no. 1, pp. 1-12, 2006.
- [21] S. Gueron and O. Zuk, “Applications of the Montgomery exponent,” *Proceedings of 2005 International Conference on Information Technology: Coding and Computing*, vol. 1, April 2005, pp. 620-625.
- [22] B. Cao, T. Srikanthan, and C.-H. Chang, “A new design method to modulo 2^n-1 squaring,” *Proceedings of IEEE International Symposium on Circuits and Systems 2005*, vol. 1, May 2005, pp. 664-667.
- [23] C.-L. Wu, D.-C. Lou, J.-C. Lai, and T.-J. Chang, “Fast modular multi-exponentiation using modified complex arithmetic,” *Applied Mathematics and Computation*, vol. 186, no. 2, pp. 1065-1074, 2007.
- [24] W. N. Chelton and M. Benaissa, “Fast Elliptic Curve Cryptography on FPGA,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 16, no. 2, pp. 198-205, Feb. 2008.

Table 1. The procedures of 4322 using Algorithm 1.

i	j	$s_{i+i-1} + x_i * x_i$	$s_{i+j-1} + 2 * x_j * x_i + u$	u	v	s_1	s_2	s_3	s_4	s_5	s_6
1	-	0+2*2	-	0	4	4	0	0	0	0	0
	2	-	0+2*3*2+0	1	2		2				
	3	-	0+2*4*2+1	1	7			7			
									1		
2	-	7+3*3	-	1	6			6			
	3	-	1+2*4*3+1	2	6				6		
										2	
3	-	2+4*4	-	1	8				6	8	
		-	-								1
				1	8	4	2	6	6	8	1

Table 2. The procedures of 8762 using Algorithm 2.

i	j	$s_{2i-1} + x_i^2$	$s_{i+j-1} + x_i x_j + d$	u	v	s_1	s_2	s_3	s_4	s_5	s_6
1	-	0+36	-	3	6	6	0	0	0	0	0
	2	-	0+6*7+3	4	7		7				
	3	-	0+6*8+4	5	4			4			
	-	-	-	0	0					1	
2	-	4+49	-	5	3			3			
	3	-	0+7*8+5	6	1				1		
	-	-	-	0	2				7	2	1
3	-	2+64	-	6	6						
	-	-	-	0	7					6	7
						6	7	3	7	6	7

Table 3. The procedures of 9872 using Algorithm 3.

i	j	$s_{i+j} + x_j * x_i + u$	$s_{2i} + x_i * x_i + u$	$s_{2i+1} + u$	u	v	s_1	s_2	s_3	s_4	s_5	s_6
1	-	-	-	-	0		0	0	0	0	0	0
	2	0+8*7+0	-	-	5	6		6				
	3	0+9*7+5	-	-	6	8			8			
	-	-	-	-	0					6		
2	-	-	-	-	0							
	3	0+9*8	-	-	7	8				2		
	-	-	-	-							7	
3	-	-	-	-	0							0
0	-	-	0+7*7+0	12+4	4	9						
					1	6	9	6	8	2	7	0
1	-	-	16+8*8+1	16+8	8	1			1	4		
					2	4						
2	-	-	14+9*9+2	0+9	9	7					7	9
					0	9						
							9	6	1	4	7	9

Table 4. The execution processes of 7892 using Algorithm 4.

i	j	u	v	s_1	s_2	s_3	s_4	s_5	s_6
1	1	8	1	0→1	0→8				
	2	7	2		8→0	0→7→11→8			
	3	6	3			8→1	6→7		
2	1	7	2		0→2	1→8			
	2	6	4			12→2	13→14→4	0→1	
	3	5	6				10→0	6→7	
3	1	6	3			5	6		
	2	5	6				12→2	12→13→3	1
	3	4	9					12→2	5→6
				1	2	5	2	2	6

Table 5. The analyses of squaring algorithms for specific execution time weight.

Operations	Algorithms				Execution Time Weight
	1	2	3	4 (Proposed)	
SPM	$\frac{n^2 + n}{2}$	$\frac{n^2 + n}{2}$	$\frac{n^2 + n}{2}$	$\times$	11
SPA	$2n^2$	$3n^2 + 4n$	$2n^2 + 2n$	$3n^2$	2
SPS	$\frac{n^2 - n}{2}$	0	N	0	3
Assignment	$\frac{n^2 + 7n}{2}$	$7n^2 + 6n$	$2n^2 + 3n + 4$	$\frac{n^2}{2} + 3n$	1
Total	$\frac{21n^2 + 15n}{2}$	$18.5n^2 + 19.5n$	$11.5n^2 + 15.5n + 4$	$6.5n^2 + 3n$	$\times$
Extra Registers	2	6	2	2	$\times$

Table 6. The speedup ratios of the squaring algorithms.

Digits	Algorithms			
	1	2	3	4 (Proposed)
1	1	0.474	0.581	1.89
2	1	0.504	0.704	1.78
4	1	0.529	0.792	1.678
8	1	0.546	0.847	1.664
16	1	0.556	0.879	1.64
32	1	0.562	0.895	1.628
64	1	0.565	0.904	1.622
128	1	0.566	0.909	1.619
∞	1	0.568	0.913	1.615

Table 7. The numbers of different digits for squaring algorithms in Table 5.

Digits	Algorithms			
	1	2	3	4 (Proposed)
1	18	38	31	9.5
2	57	113	81	32
4	198	374	250	118
8	732	1,340	864	440
16	2,808	5,048	3,196	1,712
32	10,992	19,568	12,276	6,752
64	43,488	77,024	48,100	26,816
128	172,992	305,600	190,404	106,880
∞	10.5	18.5	11.5	6.5

				x_4	x_3	x_2	x_1
$*$				x_4	x_3	x_2	x_1
				s_{14}	s_{13}	s_{12}	s_{11}
		s_{24}		s_{23}	s_{22}	s_{21}	
		s_{34}	s_{33}	s_{32}	s_{31}		
$+$	s_{44}	s_{43}	s_{42}	s_{41}			
				$2 s_{14}$	$2 s_{13}$	$2 s_{12}$	s_{11}
			$2 s_{24}$	$2 s_{23}$	s_{22}		
$+$	s_{44}	$2 s_{34}$	s_{33}				
s_8	s_7	s_6	s_5	s_4	s_3	s_2	s_1

Figure 1 The standard squaring procedures.